Guide junior : dØboguer et modifier TaskFlow

But : savoir oø regarder quand une fonctionnalitØ Øchoue, comprendre le trajet complet d'une requŒte et rØaliser une petite modification sans casser le projet.

RŁgle principale : ne modifie pas plusieurs choses au hasard. Reproduis le problŁme, lis le message exact, formule une hypothŁse, change la plus petite chose possible, teste, puis commit une modification cohØrente.

1. La carte mentale du projet

Navigateur -> React/Vite -> Nginx -> FastAPI -> SQLAlchemy -> MySQL.

React affiche les pages et envoie des requŒtes HTTP. Nginx sert les fichiers React compilØs et transmet les appels /api/v1. FastAPI applique les rŁgles mØtier, la sØcuritØ et la validation. SQLAlchemy traduit les objets Python en requŒtes paramØtrØes. MySQL conserve les utilisateurs, projets et tâches.

Dans Docker Compose, frontend, backend et db sont trois conteneurs. Si l'interface est visible mais les donnØes ne chargent pas, il faut suivre cette chaîne dans l'ordre : navigateur, frontend, proxy Nginx, backend, base de donnØes.

2. MØthode de dØbogage en six Øtapes

1. Reproduire : note les actions exactes, l'URL, le compte utilisØ et le rØsultat attendu.

2. Observer : ouvre les outils dØveloppeur du navigateur (Network et Console), les logs Docker et les messages d'erreur FastAPI.

3. Isoler la couche : interface, requŒte rØseau, API, authentification, validation, migration ou base de donnØes.

4. VØrifier l'entrØe et la sortie : statut HTTP, corps JSON, en-tŒtes Authorization/CORS, donnØes envoyØes au backend.

5. Corriger le plus petit ØlØment responsable, puis exØcuter le test ciblØ avant toute suite complŁte.

6. EmpŒcher la rØgression : ajoute ou adapte un test, documente le comportement et fais un commit clair.

3. Comprendre une requŒte backend

Exemple : POST /api/v1/projects/3/tasks. La requŒte arrive dans main.py, puis FastAPI choisit la route dans api/routes/tasks.py.

Avant la route, deps.py fournit une session SQLAlchemy et get_current_user. Cette fonction lit le Bearer token, valide le JWT, rØcupŁre l'utilisateur, sinon renvoie 401.

La route rØcupŁre le projet avec get_owned_project. Le filtre owner_id garantit que le projet appartient àl'utilisateur connectØ. Une autre personne reçoit 404.

TaskCreate dans schemas/task.py valide les donnØes. Le modŁle Task dans models/task.py

dØcrit la table. La route ajoute l'objet, commit, refresh, puis renvoie TaskResponse en JSON.

Pour une erreur backend, commence par le statut HTTP : 401 signifie token invalide ou absent; 404 peut signifier ressource absente ou non autorisØe; 422 signifie donnØes invalides; 500 signifie exception serveur àlire dans les logs.

4. Fichiers backend : oø modifier quoi ?

Nouvelle route API : backend/app/api/routes/*.py, puis inclure le routeur dans main.py si c'est un nouveau module.

Nouvelle validation ou champ de requŒte/rØponse : backend/app/schemas/*.py. Les schØmas Pydantic protŁgent l'API des entrØes inattendues.

Nouvelle colonne/table : backend/app/models/*.py, puis crØer une migration Alembic dans backend/alembic/versions. Ne modifie pas seulement le modŁle : sans migration MySQL ne connaît pas la colonne.

RŁgle mØtier rØutilisable, par exemple calcul de statistiques : backend/app/services/*.py.

Configuration : backend/app/core/config.py et .env.example. Ne mets jamais un secret dans le code ou dans Git.

Authentification : backend/app/core/security.py crØe les tokens et hash les mots de passe; backend/app/api/deps.py vØrifie le token et fournit l'utilisateur courant.

5. Exemple de modification backend : ajouter un champ àTask

Supposons que l'on demande un champ estimate_hours. Avant de coder, dØcider : type numØrique ? obligatoire ? valeur minimale ? peut-il Œtre null ?

1. Ajouter la colonne dans models/task.py, par exemple un nombre dØcimal ou entier nullable selon la rŁgle.

2. GØnØrer puis relire une migration Alembic. Elle doit ajouter la colonne sans supprimer les donnØes existantes.

3. Ajouter le champ àTaskCreate, TaskUpdate et TaskResponse dans schemas/task.py. Utiliser une contrainte comme ge=0 pour refuser une estimation nØgative.

4. Les routes de crØation/mise àjour utilisent model_dump : vØrifier que le nouveau champ est bien acceptØ et renvoyØ.

5. Ajouter des tests : crØation valide, valeur nØgative retourne 422, mise àjour, et contrôle qu'un autre utilisateur reste bloquØ.

6. Appliquer la migration, lancer les tests et vØrifier le champ via Swagger avant de modifier React.

6. Comprendre le frontend

frontend/src/main.jsx dØmarre React. App.jsx dØfinit les routes login, register et dashboard. ProtectedRoute protŁge le tableau de bord quand il n'y a pas de token.

auth.jsx contient le contexte d'authentification : jeton, connexion et dØconnexion. api.js est le point unique pour fetch : URL API, JSON, Bearer token et ApiError.

pages/Login.jsx et Register.jsx envoient les formulaires. pages/Dashboard.jsx gŁre projets, tâches, filtres, formulaires, chargement et erreurs. components.jsx contient les ØlØments rØutilisables.

Le frontend ne doit pas dØcider seul si une action est autorisØe. Il peut cacher un bouton pour une bonne expØrience utilisateur, mais seul le backend applique rØellement l'autorisation.

7. Exemple de modification frontend : afficher estimate_hours

AprŁs la modification backend, ajoute estimate_hours dans emptyTask de Dashboard.jsx.

Ajoute un input number au formulaire. Convertis proprement la valeur : un champ vide doit souvent devenir null, pas 0 par accident.

Quand saveTask construit body, ajoute estimate_hours. Affiche ensuite la valeur dans la carte de tâche seulement si elle existe.

Teste trois cas dans le navigateur : crØation avec valeur, crØation sans valeur, modification. VØrifie dans l'onglet Network que le JSON contient la valeur attendue.

Si l'interface montre undefined, compare le nom du champ envoyØ par React avec TaskResponse dans Swagger. Les noms doivent correspondre exactement.

8. DØboguer React et les appels API

Erreur dans le navigateur : ouvrir F12, Console. Une erreur JavaScript indique souvent une variable null, un import faux ou une rØponse inattendue.

DonnØes non chargØes : dans Network, sØlectionner la requŒte. VØrifier URL, mØthode, code HTTP, requŒte JSON et rØponse JSON.

401 : supprimer le token local, se reconnecter et vØrifier que api.js ajoute Authorization: Bearer <token>.

CORS : vØrifier FRONTEND_ORIGIN dans .env, recrØer le backend, et vØrifier l'origine exacte du navigateur. http://localhost:5173 et http://localhost:8080 sont diffØrentes.

422 : lire error.details dans la rØponse; il indique le champ et la validation qui a ØchouØ.

Pour dØboguer temporairement, console.log est acceptable localement, mais retirer les logs de donnØes sensibles avant de commit.

9. Docker : symptômes frØquents

Voir l'Øtat : docker compose ps. Lire les logs : docker compose logs --tail=100 backend, puis frontend ou db.

Backend unhealthy : appeler http://localhost:8001/health. Lire les logs pour migration Alembic, base non disponible, variable d'environnement manquante ou erreur Python.

Frontend ne dØmarre pas : docker compose logs frontend. Un upstream backend introuvable est souvent normal tant que le backend redØmarre; corriger d'abord le backend.

Script shell 'no such file or directory' alors que le fichier existe : penser aux fins de ligne Windows CRLF. Les scripts Linux doivent Œtre LF; .gitattributes les impose.

Build npm ci Øchoue : package.json et package-lock.json ne sont pas synchronisØs. RegØnØrer le lockfile, le tester avec npm ci, puis commit les deux fichiers cohØrents.

Ne pas utiliser docker compose down -v sauf si tu acceptes de supprimer les donnØes MySQL locales.

10. Base de donnØes et migrations

MySQL contient les donnØes persistantes dans le volume mysql_data. RedØmarrer docker compose sans -v garde le volume.

Alembic compare l'historique des migrations avec la table alembic_version. Au dØmarrage du backend, alembic upgrade head applique les migrations non exØcutØes.

AprŁs une modification de modŁle, crØer une migration, lire le fichier gØnØrØ, appliquer la migration sur une base locale et tester. Une migration mauvaise peut casser une base de production, donc ne l'exØcute jamais aveuglØment.

Pour inspecter une migration : docker compose exec backend alembic current. Pour voir une erreur SQL : docker compose logs backend et docker compose logs db.

11. Tests : choisir le bon test

Test unitaire : teste une petite fonction isolØe, par exemple le calcul des statistiques ou le hash du mot de passe.

Test API : utilise TestClient pour simuler un client HTTP, vØrifier statuts, JSON, authentification, validation et isolement entre utilisateurs.

Test frontend : vØrifie une interaction utilisateur, par exemple connexion et affichage du dashboard, avec fetch simulØ.

Smoke test : appelle le systŁme rØellement dØmarrØ par Docker, donc Nginx, backend et MySQL ensemble.

Quand tu corriges un bug, Øcris d'abord un test qui Øchoue àcause du bug si possible; applique le correctif; vØrifie que le test passe.

12. Commandes àconnaître

Stack : docker compose up -d --build ; docker compose ps ; docker compose logs -f backend ; docker compose down.

Backend : backend\.venv\Scripts\python.exe -m pytest backend\tests -q ; backend\.venv\Scripts\python.exe -m ruff check backend.

Frontend, depuis frontend : npm ci ; npm run lint ; npm run test ; npm run build.

SantØ : Invoke-RestMethod http://localhost:8001/health ; Swagger : http://localhost:8001/docs ; application : http://localhost:8080.

Git avant chaque changement : git status. AprŁs les tests : git add <fichiers>, git commit -m 'type: message', git push origin <branche>.

13. Exercices avant l'entretien

Exercice 1 : ajoute une rŁgle qui interdit un titre de tâche composØ uniquement d'espaces. Trouve le schØma, ajoute un test 422, puis vØrifie Swagger.

Exercice 2 : ajoute un filtre par date d'ØchØance. Commence par la requŒte SQLAlchemy, puis paramŁtre FastAPI, test API, puis contrôle React.

Exercice 3 : casse volontairement JWT_SECRET_KEY dans .env, observe le rØsultat, puis restaure-le. Explique pourquoi les anciens tokens deviennent invalides.

Exercice 4 : crØe deux utilisateurs, puis utilise Swagger avec le token du second pour demander un projet du premier. Explique le 404.

Exercice 5 : explique sans Øcran la diffØrence entre React, Vite, Nginx, FastAPI, SQLAlchemy, Alembic et MySQL.

14. RØponses courtes pour dØfendre le projet

Pourquoi FastAPI ? API REST lisible, validation Pydantic, injection de dØpendances, Swagger automatique et bonne intØgration SQLAlchemy/Alembic.

Pourquoi React/Vite ? L'application est un tableau de bord interactif; React organise l'Øtat et les composants, Vite fournit un dØveloppement rapide et un build statique simple.

Comment Øviter l'injection SQL ? SQLAlchemy produit des requŒtes paramØtrØes; le code n'assemble pas les entrØes utilisateur dans du SQL brut.

Comment protØger les donnØes ? JWT identifie l'utilisateur, puis les requŒtes filtrent les projets/tâches par propriØtaire. Authentification n'est pas autorisation.

Comment livrer de façon fiable ? Docker Compose reproduit la stack; migrations versionnent le schØma; tests et scans CI bloquent une release dØfaillante; les tags permettent le rollback.